6.3900 CHEAT SHEET

Training Error	Validation Error	Testing Error	
low	high	—	indicates overfitting → Solution: more training data, Ridge Regression
low/high	—	high/low	testing + training set not drawn from same distribution
high	high	—	underfitting/not expressive enough

Regression: $X: d \times n$; $Y: 1 \times n$; $\theta: d \times 1$

$\tilde{X} = \begin{bmatrix} x_1^{(1)} & \dots & x_d^{(1)} \\ \vdots & & \vdots \\ x_1^{(n)} & \dots & x_d^{(n)} \end{bmatrix}$ (d features, n data points) $\quad \tilde{Y} = \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(n)} \end{bmatrix}$ (If includes θ_0, $(d+1) \times 1$)

$\tilde{X}: n \times d$ $\quad \tilde{Y}: n \times 1$

Training Error: $J(\theta) = \frac{1}{n}(\tilde{X}\theta - \tilde{Y})^T(\tilde{X}\theta - \tilde{Y})$

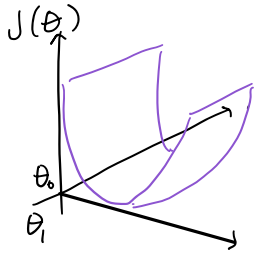
Analytical Solution → If θ^* exists, guaranteed to be unique minimizer of $J(\theta)$.

$\theta^* = (\tilde{X}^T\tilde{X})^{-1}\tilde{X}^T\tilde{Y}$ ($d \times d$, $d \times n$, $n \times 1$; has to be $d \times 1$!) → Same as $\theta^* = (XX^T)^{-1}XY^T$ ($d \times d$, $d \times n$, $n \times 1$; $d \times 1$)

$(\tilde{X}^T\tilde{X})$ → invertible if $\det \neq 0$

$(\tilde{X}^T\tilde{X})$ may not be invertible IFF $\tilde{X}$ is not a full column rank ($①$ if $n < d$ $②$ (features) columns in $\tilde{X}$ (linearly dependent) are co-linear)

Loss function is a "half-pipe" → Infinitely many optimal hypotheses/hyperplanes if $\lambda \to \infty, \|\theta\| \to 0$



Bad ! if $\lambda < 0$, $\|\theta\| \to \infty$

Solution: Regularization

Steep slope → error amplification (small change in input leading to big change in output)

Ridge Regression Regularization: $J_{ridge}(\theta, \theta_0) = \frac{1}{n}\sum_{i=1}^{n}(\theta^T x^{(i)} + \theta_0 - y^{(i)})^2 + \underbrace{\lambda\|\theta\|^2}_{\lambda > 0}$

Hyperparameter λ Purposes:

- Ensures invertible matrix
- Improves generality of model by discouraging magnitude of learned parameters from ballooning (solves error amplification issue)

Note: No need to regularize θ_0 (let the regression surface to be at its right level)

Cross-Validation:

Cross-Validate(D, k)

1 divide D into k chunks $D_1, D_2, \dots, D_k$ (of roughly equal size)
2 for i = 1 to k:
3 train h_i on $D \setminus D_i$ (withholding chunk D_i as validation set)
4 compute "test" error $\mathcal{E}_i(h_i)$ on withheld data D_i
5 return $\frac{1}{k}\sum_{i=1}^{k}\mathcal{E}_i(h_i)$

Linear Regressor Notation → $h(x; \underbrace{\theta, \theta_0}_{\text{parameters}}) = \underset{1\times2}{\theta^T} \underset{2\times1}{x} + \underset{1\times1}{\theta_0}$

OR $h(x; \theta) = \underset{1\times3}{\theta^T} \underset{3\times1}{x^{(i)}}$ (appended with θ_0; appended with constant 1 as a column ↳ a feature)

Gradient Descent:

Gradient vector → (same shape as input vector)

For $f: \mathbb{R}^m \mapsto \mathbb{R}$, $\nabla f: \mathbb{R}^m \mapsto \mathbb{R}^m$ defined at point $p = (x_1, \dots, x_m)$ in m-dimensional space

$\nabla f(p) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(p) \\ \vdots \\ \frac{\partial f}{\partial x_m}(p) \end{bmatrix}$ (SGD picks a random one)

Convex + Small enough step size + gradient descent → Guarantees convergence to global min. (if there ∃ one)

$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla_\theta f(\theta^{(t)})$

* If not convex → can get stuck at local min.
* If step size too big → can diverge

Solution: SGD (Stochastic gradient descent)

Stochastic Gradient Descent (SGD): Randomly selecting i from $\{1, \dots, n\}$

$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla_\theta f_i(\theta^{(t)})$ more random → can be "wild"

GD ⇝ SGD

Classification: linear classifier → $h(x; \theta, \theta_0) = \text{sign}(\theta^T x + \theta_0) = \begin{cases} + & \text{if } \theta^T x + \theta_0 > 0 \\ - & \text{if otherwise} \end{cases}$

linear binary classification → 2 possible labels linear separator → line separating + & − labels

Normal vector → Perpendicular to separator (points to + labels) $\theta: [x_1 \ x_2]^T$

Linear Logistic Regression: Increasing θ → steeper σ function → more sensitive (needs less to be classified a certain way)

1. calculate $z = \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_d x_d + \theta_0$
2. "squish" z with sigmoid/logistic function $g = \sigma(z) = \frac{1}{1+e^{-z}}$ Note: output always sandwiched between 0 & 1
3. Predict + class if $g > 0.5$, − otherwise.

NLL-Loss/Cross-Entropy Loss: Negative log likelihood (NLL)

$\mathcal{L}_{NLL}(g^{(i)}, y^{(i)}) =$

-(actual · log(guess) + (1-actual) · log(1-guess))

$-(y^{(i)} \cdot \log(g^{(i)}) + (1-y^{(i)})\log(1-g^{(i)}))$ ↩ same

θ_0 → shifts function horizontally (If $\theta > 0$ (θ is positive) $+\theta_0$ → left, $-\theta_0$ → right; If $\theta < 0$ (θ is negative) $+\theta_0$ → right, $-\theta_0$ → left)

Linear Separability:

Potentially helps → more features

Hinders linear separability → more training data

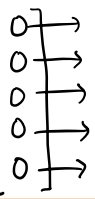
Not helpful features:
- Globally-"same" feature (constants)
- Linearly dependent on existing feature(s)

Multi-class Classification:

Softmax Function: (sum up to 1)

$$y = \text{softmax}(z) = \begin{bmatrix} e^{z_1}/\sum_i e^{z_i} \\ \vdots \\ e^{z_k}/\sum_i e^{z_i} \end{bmatrix}$$

Softmax (5 classes)

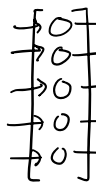


$-\sum_{j=1}^{5} y_j \log(g_j)$

Uses NLLM

choose max value as class

Multiple sigmoids (5 classes)



$\sum_{j=1}^{5} -[y_j \log(g) + (1-y_j)\log(1-g)]$

Uses NLL

Negative Log Likelihood Multi-class (NLLM):

$\mathcal{L}_{nllm}(g, y) = -\sum_{k=1}^{K} y_k \cdot \log(g_k)$ (K classes)

$\nabla_\theta \mathcal{L}_{nllm}(g, y) = x(g - y)^T$

Confidence/Prediction Accuracy:

$A(h; D) = 1 - \frac{1}{n}\sum_{i=1}^{n} \mathcal{L}_{01}(g^{(i)}, y^{(i)})$

Vector $g \perp$ to separator: $g = \theta/(\sqrt{\theta^T\theta})$

$\|\theta\| = \sqrt{\theta^T\theta}$

Distance from point x to separator: $\frac{|\theta^T x + \theta_0|}{\sqrt{\theta^T\theta}}$

Confidence: $\frac{|\theta^T x + \theta_0|}{R\|\theta\|}$

R → distance between 2 points farthest from each other in data set

One-vs-All (OVA):

plane m with arrowing pointing to halfspace predicting positive m.

$h_{ova}(x)$: Consider only set of classifiers in P_{ova} predicting +1 for x. Predict x as class with largest confidence value. (furthest one)

One-vs-One (OVO): (Lower error / better for classes close to each other)

plane lm with arrow pointing into halfspace predicting m

$h_{ovo}(x)$: Consider the classifier between classes l & m.
If class l at x, vote for l. If class m at x, vote for m.
Predict class with most votes.

Features: $\theta^T \phi(x) + \theta_0$ (feature transformation)

If order too high → overfitting, too long to calculate

Polynomial Basis Construction → adding polynomial term (ex. $h(x) = \theta_2 x^2 + \theta_1 x + \theta_0$)

$$h(x) = \sum_{i=1}^{k} \theta_i x^i = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \vdots \\ \theta_k \end{bmatrix} \cdot \underbrace{\begin{bmatrix} 1 \\ x \\ x^2 \\ \vdots \\ x_k \end{bmatrix}}_{\text{polynomial basis}}$$

Total # of features = $\binom{d+k}{k} = \frac{(d+k)!}{d!k!}$

Picking k → Use cross-validation or validation

Radial Basis Functions → β influence "broad" of influence of each point.

$f_p(x) = e^{-\beta\|p - x\|^2}$

Smaller β → more general, larger "neighborhood"

larger β → more specific, too large may lead to overfitting

Encoding Features: Numerical, one-hot, thermometer, factored, standardization

Just like one-hot encoding, but grouping categories that relate to each other.

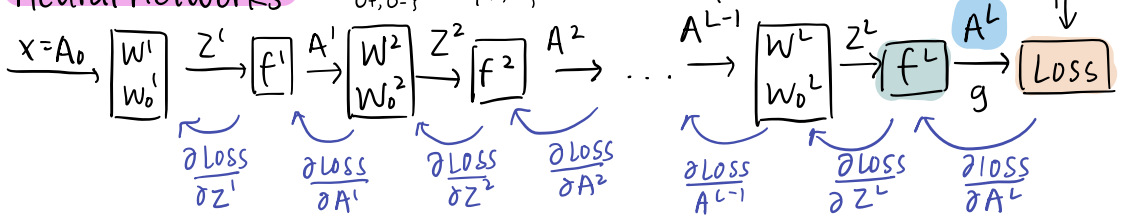
$\phi(x_i) = \frac{x_i - \bar{x}_i}{\sigma_i}$ $\bar{x}_i :=$ mean$_i$, $\sigma_i :=$ standard deviation$_i$

standard deviation: (average squared distance from mean)

$\sigma = \sqrt{\sum_i \frac{(x-\bar{x})^2}{n}}$

Neural Networks:

ex. Blood Types {A+, A-, B+, B-, AB+, AB-, O+, O-} → {A, B, AB, O} → {+, -}



Error back-propagation: Gradient descent + Chain Rule.

- Forward pass to compute all the a & z values at all the layers, & finally the actual loss.
- Work backwards to compute gradient of loss with respect to weights in each layer (from layer L to 1)

$$\frac{\partial Loss}{\partial Z^{(l)}} = \frac{\partial A^{(l)}}{\partial Z^{(l)}} \frac{\partial Z^{(l+1)}}{\partial A^{(l)}} \frac{\partial A^{(l+1)}}{\partial Z^{(l+1)}} \cdots \frac{\partial A^{(L-1)}}{\partial Z^{(L-1)}} \frac{\partial Z^{(L)}}{\partial A^{(L-1)}} \frac{\partial A^{(L)}}{\partial Z^{(L)}} \frac{\partial Loss}{\partial A^{L}}$$

Activation Function Choices: ReLU, sigmoid, hyperbolic tangent, softmax

common for output for binary classification

common for output of multi-class classification

Rectified linear unit (ReLU)

$$ReLU(z) = \begin{cases} 0 & \text{if } z < 0 \\ z & \text{otherwise} \end{cases} = \max(0, z)$$

Common in internal layers

$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$ range $(-1, 1)$

Matching Activation & Loss Functions:

Loss	f^L	task
squared	linear	regression
NLL	sigmoid	binary classification
NLLM	Softmax	multi-class classification

Back Propagation example: let $E \to$ error/loss

$X \xrightarrow{W_1}$ hidden unit (ReLU) $\xrightarrow{W_2}$ output unit (Sigmoid) $\to y_1$

$$\frac{\partial E}{\partial W_1} = \frac{\partial E}{\partial y} \cdot \frac{\partial y}{\partial z_2} \cdot \frac{\partial z_2}{\partial a} \cdot \frac{\partial a}{\partial z_1} \cdot \frac{\partial z_1}{\partial W_1}$$

$\frac{\partial E}{\partial y} = y - t$ $\quad \frac{\partial y}{\partial z_2} = y(1-y)$ (derivative of sigmoid) $\quad \frac{\partial z_2}{\partial a} = W_2$ $\quad \frac{\partial a}{\partial z_1} = \begin{cases} 1 & \text{if } W_2 x > 0 \\ 0 & \text{if } W_2 x < 0 \end{cases}$ (derivative of ReLU) $\quad \frac{\partial z_1}{\partial W_1} = X$

Update: $W_1 := W_1 - \eta \frac{\partial E}{\partial W_1}$

Expressitivity:

- hidden units add expressitivity to the network, hidden layers do not.

CNN: characteristics of identifying images → visual hierarchy, spatial locality, translational invariance

- Padding, # of filters, filter size, stride — figure out through padding + stride.
- Applying filters: dot product — output dimensions: ? × ? × # of filters (depth dimension)
- **Max-pooling**: size $(k \times k)$, stride, no padding
  typical "pyramid" stack → image sizes get smaller (want to find local pattern) in layers of processing

fully connected layer: # of weights (parameters) → (# of input + 1) · (# of output) (↑ offset weight)

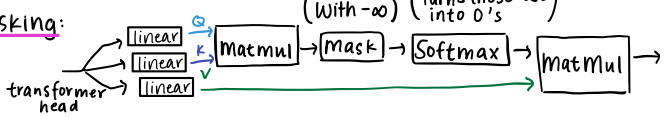
Transformers:

1. tokenization: tokens → vector of neurons (teams / people) $\begin{cases} d: \text{size of each token} \\ n: \text{\# of tokens} \end{cases}$
2. attention mechanisms — Self-Attention $(Q, K, V \in \mathbb{R}^{n \times d_k})$

query d_q keys d_k values d_v

$Q = XW_q \quad K = XW_k \quad V = XW_v$

Masking: (with $-\infty$) (softmax turns those $-\infty$'s into 0's)

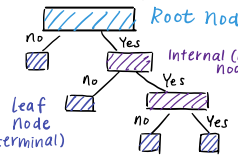


Single-Query: (i enumerates over queries, j enumerates over keys) — multi-query

1. Similarity score with key j: $s_j = (q^T k_j)/\sqrt{d_k}$ $\quad s_{ij} = (q_i^T k_j)/\sqrt{d_k}$
2. Attention Weights (softmax'ed scores)
   $a = \text{softmax}([q^T k_1 \ \dots \ q^T k_n]/\sqrt{d_k})$ $\quad A = \begin{bmatrix} a_1 \\ \vdots \\ a_{n_q} \end{bmatrix}$
   $a_i = \text{Softmax}([s_{i1} \ s_{i2} \ \dots \ s_{in_k}])$ Stack all such a_i vertically
3. output: attention-weighted sum $y = \sum_j a_j v_j$
   $y_i = \sum_j a_{ij} v_j \to$ stack vertically $(y \in \mathbb{R}^{n_q \times d_v})$

Non-Parametric Models: Decision trees, k-Nearest Neighbor

Decision Tree (flow chart)



For each internal node:
- split dimension
- split value
- child nodes

Note: no work training, all work predicting

k-Nearest Neighbors:
Find k training points near query point x and output majority y value (classification) OR locally weighted regression

Weighted Average Entropy: metric for describing purity/chaos among data (low ↓ / high ↓)

N_m := # of data points total

$$E_{j,s} = \frac{|I_{j,s}^-|}{N_m} \cdot H(I_{j,s}^-) + \frac{|I_{j,s}^+|}{N_m} \cdot H(I_{j,s}^+)$$

(fraction of data in left data set; fraction of data in right data set)

$H = -\sum_{\text{classes}} \hat{P}_c \log_2 \hat{P}_c$ (Entropy)

$\hat{P}_c$ = (# of points in that class) / (total # of points)

Algorithm for building tree:

Input I: set of indices
k: hyper-parameter, maximum node/leaf "size"
$\hat{y}$: (intermediate) prediction
j: split dimension
s: split value

(# of data points that will arrive towards a leaf)

If size of input $I > k$ $(|I| > k)$:

for each internal node (j, s) (each $j \in I$, each s):

$I_{j,s}^+ = \{i \in I \mid x_j^{(i)} \geq s\}$ (Points to the right side of the split)

$I_{j,s}^-$ (points to the left side of the split)

$\hat{y}_{j,s}^+$ Empirical average of the points (to the right)

$\hat{y}_{j,s}^-$ same, but to the left

Regression: Average
Classification: Majority Vote

$E_{j,s} = \sum_{i \in I_{j,s}^+} (y^{(i)} - \hat{y}_{j,s}^+)^2 + \sum_{i \in I_{j,s}^-} (y^{(i)} - \hat{y}_{j,s}^-)^2$ (MSE)

Considers a finite # of (j,s) combo suffices (those in-between data points)

$(j^*, s^*) = \text{argmin}_{j,s} E_{j,s}$ ← Picks the "best" among these combos (ties broken arbitrarily)

else:

$\hat{y} = \text{average}_{i \in I} y^{(i)}$

return LEAF(leaf_value = $\hat{y}$)

(Deals with leaf nodes)

return Node$(j^*, s^*$, BuildTree(I_{j^*,s^*}^-, k), BuildTree$(I_{j^*,s^*}^+, k))$ ← Recursion of building further down the tree

MDP:

Value Function: $V_\pi^h(s) = \mathbb{E}\left[\sum_{t=0}^{h-1} \gamma^t R(s_t, \pi(s_t)) \mid s_0 = s, \pi\right], \forall s, h$

expected sum of discounted rewards, for starting in state s, following policy π, over h horizons.

(Optimal) State-Action Value Function $Q^h(s,a)$:

$Q^h(s,a) = R(s,a) + \gamma \sum_{s'} T(s,a,s') \max_{a'} Q^{h-1}(s', a'), \forall s, a$

$\pi_h^*(s) = \text{argmax}_a Q^h(s,a), \forall s, h$ (optimal action)

$Q^h(s,a) = R(s,a) + \gamma \sum_{s'} T(s,a,s') \max_{a'} Q^{h-1}(s', a')$ (Infinite Horizon)

Bellman Recursion:

$V_\pi^h(s) = R(s, \pi(s)) + \gamma \sum_{s'} T(s, \pi(s), s') V_\pi^{h-1}(s')$

(immediate reward; state s, taking action given by policy $\pi(s)$; discount; weighted by probability of getting to s'; (h-1) horizon values at next state s')

Bellman Equation: (infinite-horizon)

$V_\pi(s) = R(s, \pi(s)) + \gamma \sum_{s'} T(s, \pi(s), s') V_\pi(s'), \forall s$

Geometric Series for finding V*(s) with gamma (γ): (constant reward for t steps until goal state)

$\sum_{i=0}^{n} \gamma^i = \frac{1 - \gamma^{n+1}}{1 - \gamma}$

If reward = c until goal state, which takes t steps from s_0,

$$V^*(s_0) = c(1 + \gamma + \gamma^2 + \gamma^3 + \dots + \gamma^{t-1}) = c \sum_{i=0}^{t-1} \gamma^i = c\left(\frac{1 - \gamma^t}{1 - \gamma}\right)$$

Reinforcement Learning:

Model-based methods: keep playing game to approximate the unknown rewards & transitions

↳ **Q-Learning:**

$$Q_{new}(s,a) \leftarrow (1-\alpha) \underbrace{Q_{old}(s,a)}_{\text{old belief}} + \alpha \underbrace{(r + \gamma \max_{a'} Q_{old}(s',a'))}_{\text{target}}$$

learning rate, typically $\in [0,1]$; discount factor

If S and/or A are continuous:

$Q_{new}(s,a) \leftarrow Q_{old}(s,a) + \alpha[r + \gamma \max_{a'} Q_{old}(s',a') - Q_{old}(s,a)]$

· minimize $[Q(s,a) - (r + \gamma \max_{a'} Q(s',a'))]^2$ using gradient method.

Q-learning algorithm:

converges so long as S and A are finite

Q-learning $(S, A, \gamma, s_0, \alpha)$:

for every state s and action a in S & A:

$Q_{old}(s,a) = 0$

$s \leftarrow s_0$

while true:

$a \leftarrow$ pick action for $Q(s, Q_{old}(s,a))$ — Use ϵ-greedy strategy to pick action (or another strategy)

(good for discrete S & A)

· with probability $1-\epsilon$, choose $\text{argmax}_a Q(s,a)$ (Exploitation)

· with probability ϵ, choose an action $a \in A$ uniformly at random (Exploration)

execute action a → observe results r, s'

$Q_{new}(s,a) \leftarrow (1-\alpha) Q_{old}(s,a) + \alpha(r + \gamma \max_{a'} Q_{old}(s',a'))$

$s \leftarrow s'$

if $\max_{s,a} |Q_{old}(s,a) - Q_{new}(s,a)| < \epsilon$: (error acceptance)

return Q_{new}

$Q_{old} \leftarrow Q_{new}$

Clustering:

↳ k-mean algorithm: assign to k clusters, minimize variance

↳ objective: denote assignment as $y^{(i)} \in \{1, 2, \ldots, k\}$

$$\text{loss} = \sum_{j=1}^{k} \sum_{i=1}^{n} \mathbb{1}(y^{(i)}=j) \|x^{(i)} - \mu^{(j)}\|^2, \quad \mu^{(j)} = \frac{1}{N_j} \sum_{i=1}^{n} \mathbb{1}(y^{(i)}=j) x^{(i)}, \quad N_j = \sum_{i=1}^{n} \mathbb{1}(y^{(i)}=j)$$

(loss non-increasing for each iteration)

loss: sum of all the distances between a point and its assigned cluster mean squared. (note: distances squared first before summing)

Initialization and value of k is important!

↳ don't want cluster centers to be close to each other!

↳ k-means finds poor local optima of K-means objective

distance metric needed → algorithm uses euclidean

would not be able to recover ground truth clusters if too many points favor one side over another or points are too close to each other.

→ K-MEANS $(k, \tau, \{x^{(i)}\}_{i=1}^n)$

μ, y = random initialization

for t = 1 to τ

$y_{old} = y$

for i = 1 to n: $y^{(i)} = \text{argmin}_j \|x^{(i)} - \mu^{(j)}\|^2$ (assign to nearest cluster)

Assign to nearest cluster, and calculate mean for each cluster.

for j = 1 to k: $\mu^{(j)} = \frac{1}{N_j} \sum_{i=1}^{n} \mathbb{1}(y^{(i)}=j) x^{(i)}$ (recalculate mean)

if $\mathbb{1}(y = y_{old})$: break

return μ, y

↳ Gradient Descent: $L(u) = \sum_{i=1}^{n} \min_j \|x^{(i)} - \mu^{(j)}\|^2$, $y^{(i)} = \text{argmin}_j \|x^{(i)} - \mu^{(j)}\|^2$

Auto-encoder: Punchline → Bottleneck in middle (could be multi-layer)

x (image) —encode→ F (compressed image code $\min_\theta \|x - \tilde{x}\|^2$ (vector z)) —decode→ $\tilde{x} = F(x;\theta)$ (Reconstructed image)

$\mathbb{R}^d \to \mathbb{R}^k$ — Want k<d (compressed image smaller than original image) — $\mathbb{R}^k \to \mathbb{R}^d$

Minimize Reconstruction Error:

$$\min_{W^1, W_0^1, W^2, W_0^2} \sum_{i=1}^{n} L_{SE}\left(h(g(x^{(i)}; W^1, W_0^1); W^2, W_0^2), x^{(i)}\right)$$

Mean-Squared Error: $\frac{1}{n} \sum_{i=1}^{n} (Y_i - \hat{Y}_i)^2$

Mean-Absolute Error: $\frac{\sum_{i=1}^{n} |y_i - x_i|}{n}$

loss function doesn't penalize outliers as much as MSE

Numpy:

indexing A[i,j]

splicing A[i:i', j:j']

column vector A[:, i:i+1]

np.full((m,n), 1)

np.transpose(A) = A.T

np.linalg.inv(A)

np.linalg.mean(A, axis, keepdims=false) } axis=0 row, axis=1 column

np.linalg.norm(A, axis, keepdims)

np.sum(A, axis)

np.vstack(A) → A = sequence of arrays

np.ones((m,n)), np.zeros((m,n))

np.random.randint(n) → $\in [0, n)$

np.random.rand() → $\in [0, 1)$

Ellipse feature transformation:

$$\phi\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} x_1 \\ x_2 \\ x_1^2 \\ x_2^2 \\ x_1 x_2 \end{bmatrix}$$

$(ax_1 - b)^2 + (cx_2 - d)^2$

↓ expand

$a^2x_1^2 - 2abx_1 + b^2 + c^2x_2^2 - 2cdx_2 + d^2$

coefficients needed (x_1, x_2, x_1^2, x_2^2)

x_1x_2 : allows for rotation of ellipses' axes.